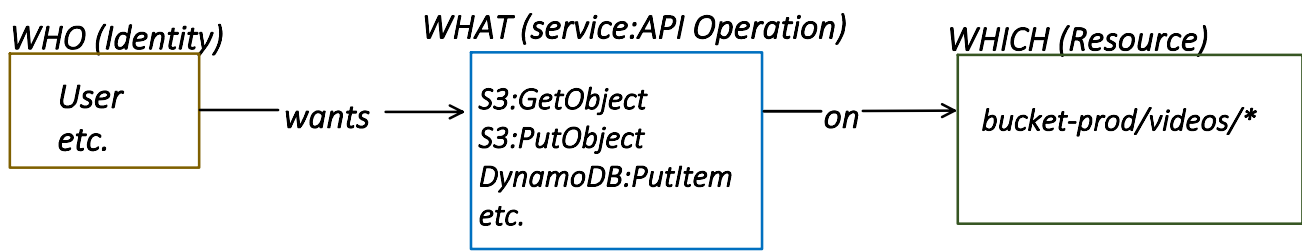


IAM (Identity and Access Management)

Every API call in AWS ask the same question:

Identity X wants to perform Action Y on Resource Z. Allow or Deny?



Like:

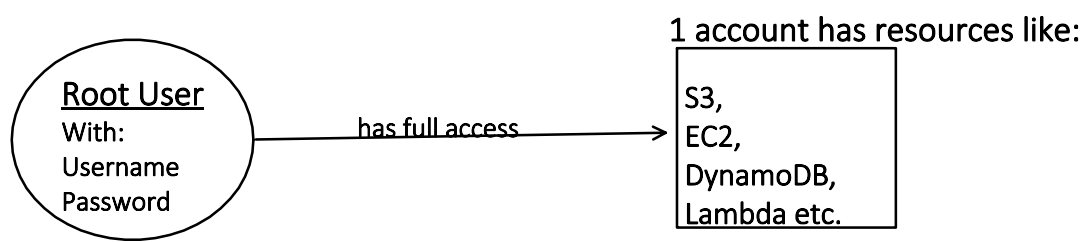
User "Shrayansh" wants to invoke S3 GetObject API on bucket-prod/videos/*. Should be allow or deny?

very Important:

In AWS, every API call is DENIED by-default.
So, If we want someone to access resource, then we have to explicitly grant the permission.

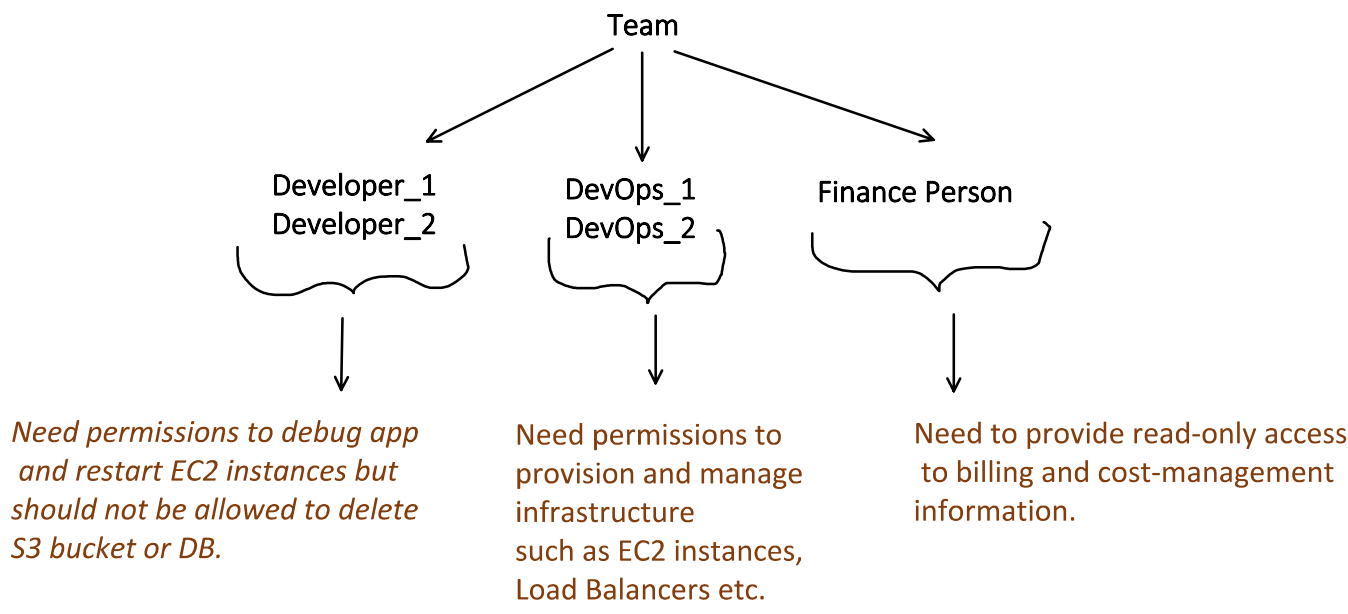
And this is where IAM comes into the picture.

Till now the account setup which we have done, looks something like this:



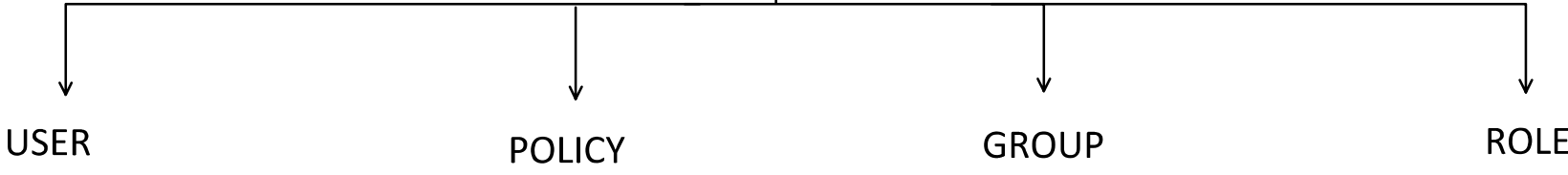
Root User has unlimited power, it even can delete the account too.

That’s why using the Root User credentials across a team is extremely risky because everyone gets unrestricted access with no proper isolation or accountability.



So, a single Root User is not sufficient for a real organization.
We need multiple identities, each with precise permissions, that can be managed independently.
and that is exactly what IAM provides.

IAM Entities:

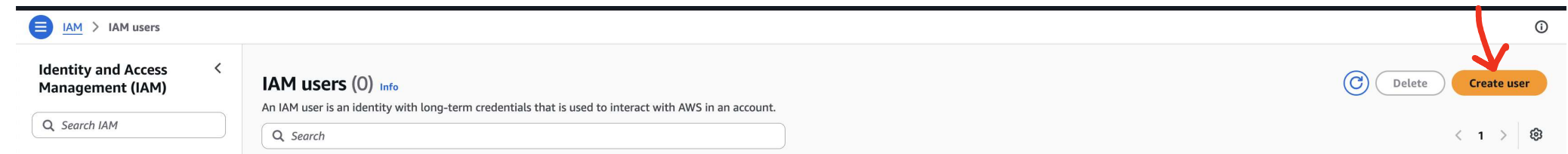


1. IAM User

- User represents either:
 - Human
 - or long lived program
- Each User has:
 - ARN (Amazon Resource Name), unique identifier for the resource.
 - Credentials:
 - Username / Password for Console login
 - Access Key ID + Secret Key (for SDK)
 - Permissions

Lets Create User:

Go to IAM -> Users -> Click Create User



Provide Username -> Click Next

IAM > IAM users > Create user

Step 1: Specify user details

Specify user details

User details

User name: Shrayansh

The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and +, =, @, _ (hyphen)

☐ Provide user access to the AWS Management Console - optional

In addition to console access, users with `SigntecLocalDevelopmentAccess` permissions can use the same console credentials for programmatic access without the need for access keys.

☐ If you are creating programmatic access through access keys or service-specific credentials for AWS CodeCommit or Amazon Keyspaces, you can generate them after you create this IAM user. [Learn more](#)

Cancel Next

Simply click Next (Group, Policy, Permission boundary will cover later in IAM)

IAM > IAM users > Create user

Step 1: Specify user details

Step 2: Set permissions

Set permissions

Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

Permissions options

☒ Add user to group

Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.

☐ Copy permissions

Copy all group memberships, attached managed policies, and inline policies from an existing user.

☐ Attach policies directly

Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

☐ Get started with groups

Create a group and select policies to attach to the group. We recommend using groups to manage user permissions by job function, AWS service access, or custom permissions. [Learn more](#)

Create group

Set permissions boundary - optional

Cancel Previous Next

User is created

IAM > IAM users > Shrayansh

Shrayansh

Summary

ARN: `arn:aws:iam::64533705313:user/Shrayansh`

Created: May 27, 2026, 14:41 (UTC+05:30)

Console access: Disabled

Last console sign-in: -

Access key 1: Create access key

Permissions policies (0)

Permissions are defined by policies attached to the user directly or through groups.

Filter by Type: All types

Policy name: Type: Attached via

No resources to display

Permissions boundary (not set)

Generate policy based on CloudTrail events

You can generate a new policy based on the access activity for this user, then customize, create, and attach it to this role. AWS uses your CloudTrail events to identify the services and actions used and generate a policy. [Learn more](#)

Generate policy

No requests to generate a policy in the past 7 days.

How this newly created User will login?

Console Username/Password

API Key + Secret Key

Click on Security Credentials

IAM > IAM users > Shrayansh

Shrayansh

Summary

ARN: `arn:aws:iam::64533705313:user/Shrayansh`

Created: May 27, 2026, 14:41 (UTC+05:30)

Console access: Disabled

Last console sign-in: -

Access key 1: Create access key

Permissions policies (0)

Permissions are defined by policies attached to the user directly or through groups.

Filter by Type: All types

Policy name: Type: Attached via

No resources to display

Permissions boundary (not set)

Generate policy based on CloudTrail events

You can generate a new policy based on the access activity for this user, then customize, create, and attach it to this role. AWS uses your CloudTrail events to identify the services and actions used and generate a policy. [Learn more](#)

Generate policy

No requests to generate a policy in the past 7 days.

Click on Enable Console access

Shrayansh

Summary

ARN: `arn:aws:iam::64533705313:user/Shrayansh`

Created: May 27, 2026, 14:41 (UTC+05:30)

Console access: Disabled

Last console sign-in: -

Access key 1: Create access key

Permissions policies (0)

Permissions are defined by policies attached to the user directly or through groups.

Filter by Type: All types

Policy name: Type: Attached via

No resources to display

Permissions boundary (not set)

Generate policy based on CloudTrail events

You can generate a new policy based on the access activity for this user, then customize, create, and attach it to this role. AWS uses your CloudTrail events to identify the services and actions used and generate a policy. [Learn more](#)

Generate policy

No requests to generate a policy in the past 7 days.

Enable console access

Console sign-in

Console sign-in link: <https://64533705313.signin.aws.amazon.com/console>

Console password: Not enabled

Multi-factor authentication (MFA) (0)

Use MFA to increase the security of your AWS environment. Signing in with MFA requires an authentication code from an MFA device. Each user can have a maximum of 6 MFA devices assigned. [Learn more](#)

Assign MFA device

Access keys (0)

Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. [Learn more](#)

Create access key

Click on Create Access Key

Shrayansh

Summary

ARN: `arn:aws:iam::64533705313:user/Shrayansh`

Created: May 27, 2026, 14:41 (UTC+05:30)

Console access: Disabled

Last console sign-in: -

Access key 1: Create access key

Permissions policies (0)

Permissions are defined by policies attached to the user directly or through groups.

Filter by Type: All types

Policy name: Type: Attached via

No resources to display

Permissions boundary (not set)

Generate policy based on CloudTrail events

You can generate a new policy based on the access activity for this user, then customize, create, and attach it to this role. AWS uses your CloudTrail events to identify the services and actions used and generate a policy. [Learn more](#)

Generate policy

No requests to generate a policy in the past 7 days.

Enable console access

Console sign-in

Console sign-in link: <https://64533705313.signin.aws.amazon.com/console>

Console password: Not enabled

Multi-factor authentication (MFA) (0)

Use MFA to increase the security of your AWS environment. Signing in with MFA requires an authentication code from an MFA device. Each user can have a maximum of 6 MFA devices assigned. [Learn more](#)

Assign MFA device

Access keys (0)

Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. [Learn more](#)

Create access key

Set your Password

Shrayansh

Summary

ARN: `arn:aws:iam::64533705313:user/Shrayansh`

Created: May 27, 2026, 14:41 (UTC+05:30)

Console access: Disabled

Last console sign-in: -

Access key 1: Create access key

Permissions policies (0)

Permissions are defined by policies attached to the user directly or through groups.

Filter by Type: All types

Policy name: Type: Attached via

No resources to display

Permissions boundary (not set)

Generate policy based on CloudTrail events

You can generate a new policy based on the access activity for this user, then customize, create, and attach it to this role. AWS uses your CloudTrail events to identify the services and actions used and generate a policy. [Learn more](#)

Generate policy

No requests to generate a policy in the past 7 days.

Enable console access

Console sign-in

Console sign-in link: <https://64533705313.signin.aws.amazon.com/console>

Console password: Not enabled

Multi-factor authentication (MFA) (0)

Use MFA to increase the security of your AWS environment. Signing in with MFA requires an authentication code from an MFA device. Each user can have a maximum of 6 MFA devices assigned. [Learn more](#)

Assign MFA device

Access keys (0)

Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. [Learn more](#)

Create access key

Save the URL, Username and Password for future.

Shrayansh

Summary

ARN: `arn:aws:iam::64533705313:user/Shrayansh`

Created: May 27, 2026, 14:41 (UTC+05:30)

Console access: Disabled

Last console sign-in: -

Access key 1: Create access key

Permissions policies (0)

Permissions are defined by policies attached to the user directly or through groups.

Filter by Type: All types

Policy name: Type: Attached via

No resources to display

Permissions boundary (not set)

Generate policy based on CloudTrail events

You can generate a new policy based on the access activity for this user, then customize, create, and attach it to this role. AWS uses your CloudTrail events to identify the services and actions used and generate a policy. [Learn more](#)

Generate policy

No requests to generate a policy in the past 7 days.

Enable console access

Console sign-in

Console sign-in link: <https://64533705313.signin.aws.amazon.com/console>

Console password: Not enabled

Multi-factor authentication (MFA) (0)

Use MFA to increase the security of your AWS environment. Signing in with MFA requires an authentication code from an MFA device. Each user can have a maximum of 6 MFA devices assigned. [Learn more](#)

Assign MFA device

Access keys (0)

Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. [Learn more](#)

Create access key

Select your reason for creating Access Key

Shrayansh

Summary

ARN: `arn:aws:iam::64533705313:user/Shrayansh`

Created: May 27, 2026, 14:41 (UTC+05:30)

Console access: Disabled

Last console sign-in: -

Access key 1: Create access key

Permissions policies (0)

Permissions are defined by policies attached to the user directly or through groups.

Filter by Type: All types

Policy name: Type: Attached via

No resources to display

Permissions boundary (not set)

Generate policy based on CloudTrail events

You can generate a new policy based on the access activity for this user, then customize, create, and attach it to this role. AWS uses your CloudTrail events to identify the services and actions used and generate a policy. [Learn more](#)

Generate policy

No requests to generate a policy in the past 7 days.

Enable console access

Console sign-in

Console sign-in link: <https://64533705313.signin.aws.amazon.com/console>

Console password: Not enabled

Multi-factor authentication (MFA) (0)

Use MFA to increase the security of your AWS environment. Signing in with MFA requires an authentication code from an MFA device. Each user can have a maximum of 6 MFA devices assigned. [Learn more](#)

Assign MFA device

Access keys (0)

Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. [Learn more](#)

Create access key

Based on your reason, it shows better alternative.

like instead of creating long lived access id + secret key, it is suggesting us to use ROLE.

Save the Access Key and Secret Access key

Shrayansh

Summary

ARN: `arn:aws:iam::64533705313:user/Shrayansh`

Created: May 27, 2026, 14:41 (UTC+05:30)

Console access: Disabled

Last console sign-in: -

Access key 1: Create access key

Permissions policies (0)

Permissions are defined by policies attached to the user directly or through groups.

Filter by Type: All types

Policy name: Type: Attached via

No resources to display

Permissions boundary (not set)

Generate policy based on CloudTrail events

You can generate a new policy based on the access activity for this user, then customize, create, and attach it to this role. AWS uses your CloudTrail events to identify the services and actions used and generate a policy. [Learn more](#)

Generate policy

No requests to generate a policy in the past 7 days.

Enable console access

Console sign-in

Console sign-in link: <https://64533705313.signin.aws.amazon.com/console>

Console password: Not enabled

Multi-factor authentication (MFA) (0)

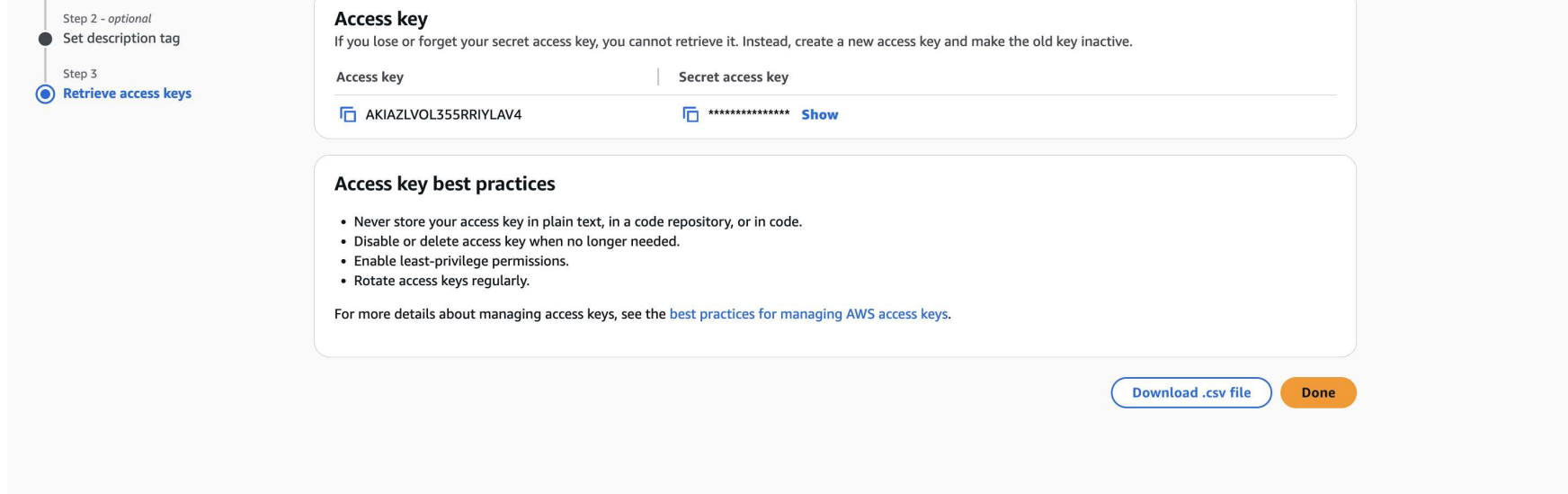
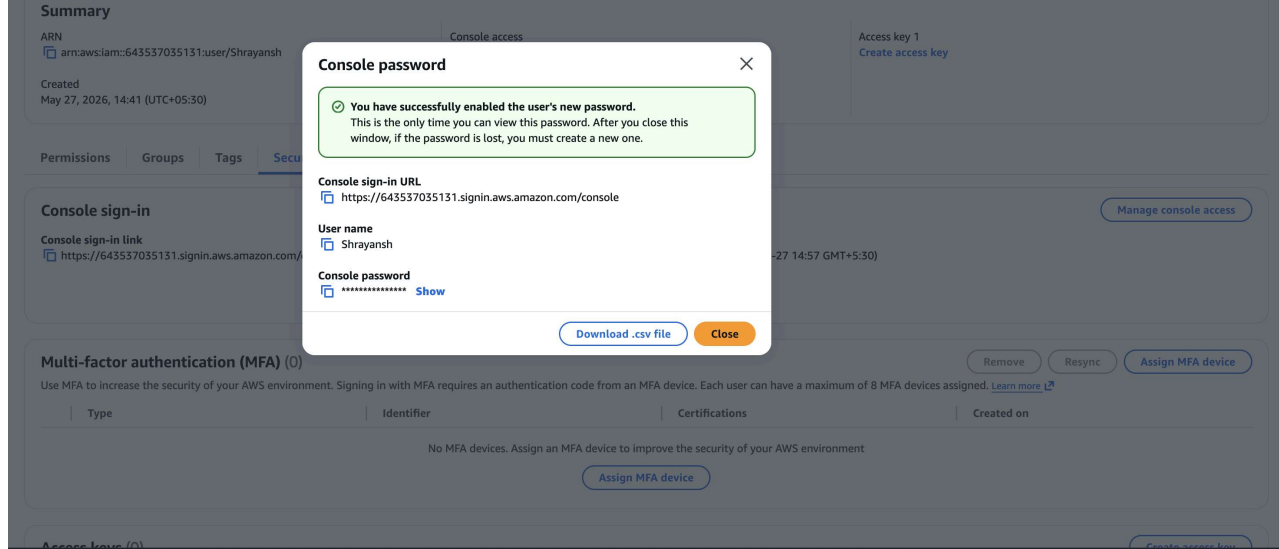
Use MFA to increase the security of your AWS environment. Signing in with MFA requires an authentication code from an MFA device. Each user can have a maximum of 6 MFA devices assigned. [Learn more](#)

Assign MFA device

Access keys (0)

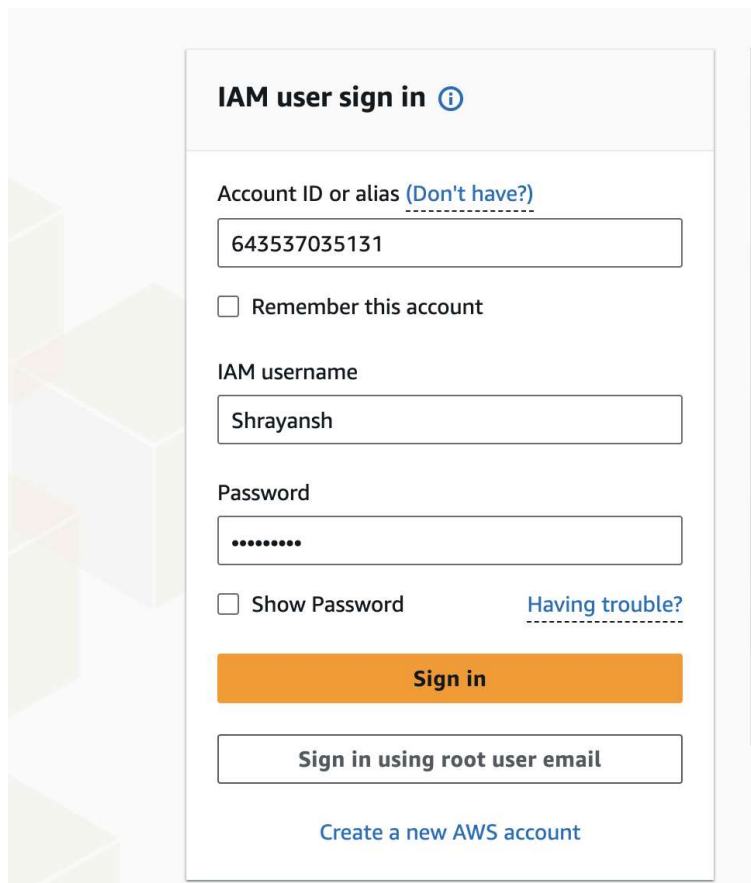
Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. [Learn more](#)

Create access key

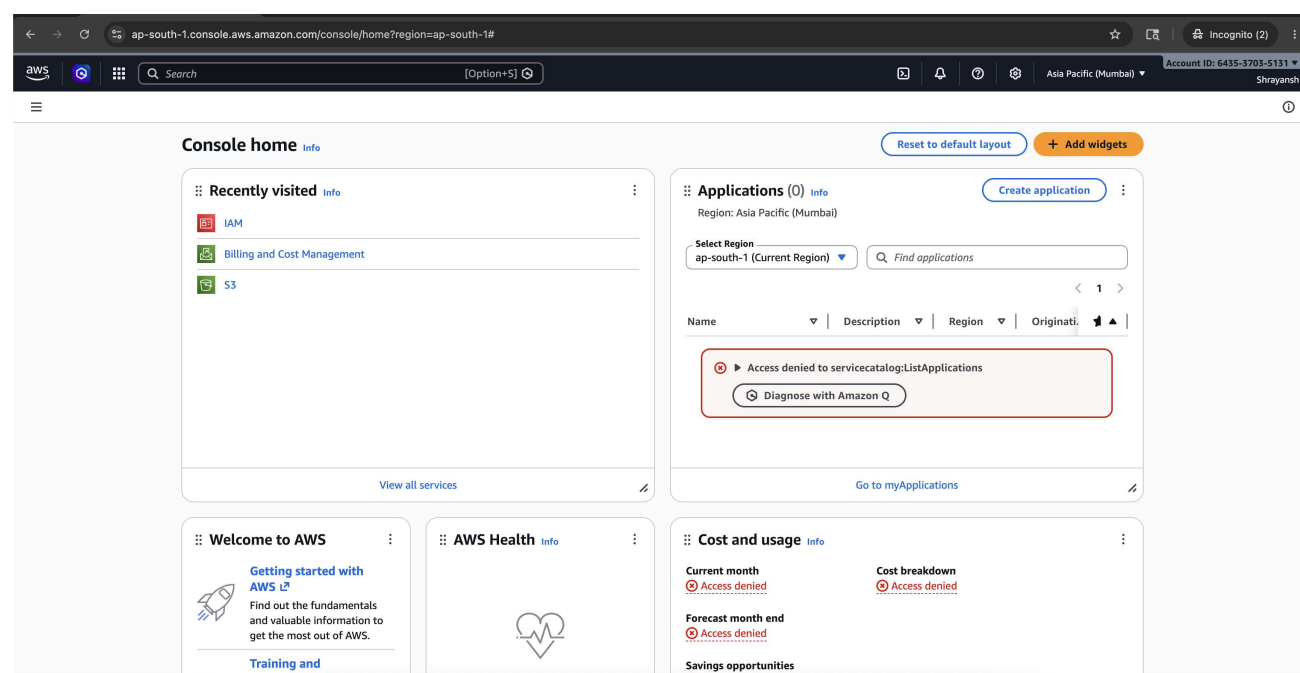


Lets Login with the newly created User credentials:

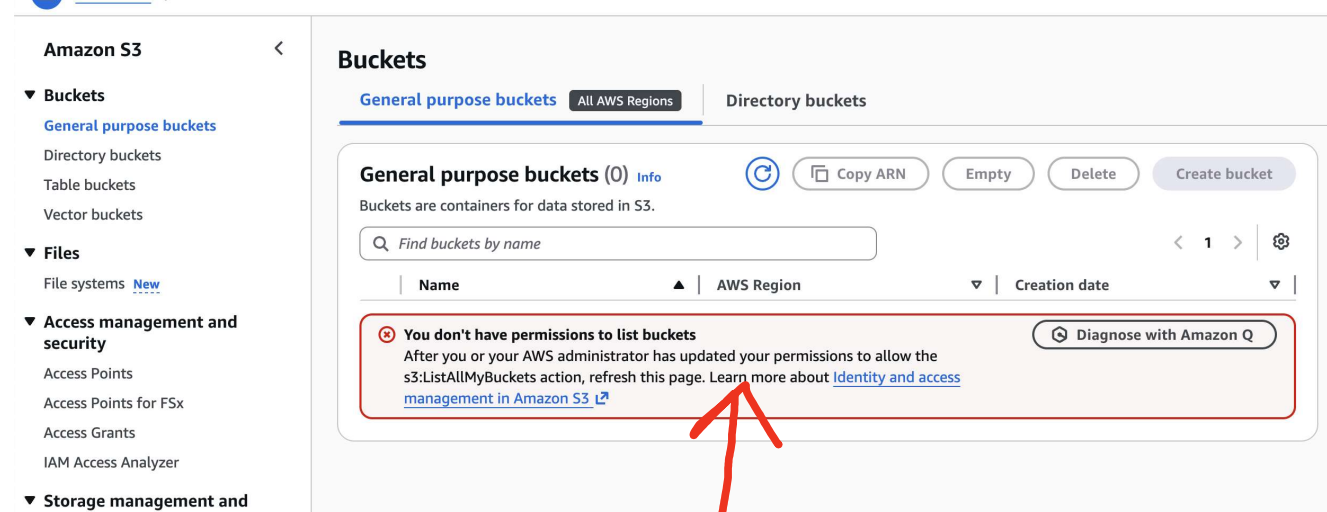
Use the Console sign-in URL, which was returned during User creation



Provide the credentials, and login successfully



If, we try to access S3 buckets, we get ACCESS DENIED



Here you will notice that, everywhere we see "Access Denied".

As I mentioned earlier, AWS API call is DENIED by-default.

So till now we have not granted any permission to this newly created user, that's why we see Access denied everywhere.

Lets use API key and Secret key from code to access AWS resources.

Config class

```
package com.concepts;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import software.amazon.awssdk.auth.credentials.AwsBasicCredentials;
import software.amazon.awssdk.auth.credentials.StaticCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;

@Configuration
public class AwsConfig {
    @Value("${aws.access-key}")
    private String accessKey;

    @Value("${aws.secret-key}")
    private String secretKey;

    @Value("${aws.region}")
    private String region;

    @Bean
    public S3Client s3Client() {
        AwsBasicCredentials credentials =
            AwsBasicCredentials.create(accessKey, secretKey);

        return S3Client.builder()
            .region(Region.of(region))
            .credentialsProvider(
                StaticCredentialsProvider.create(credentials)
            )
            .build();
    }
}
```

Reading access key id and secret id from properties, generally it should be stored in vault, this is just for testing purpose.

Controller class

```
package com.concepts;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import software.amazon.awssdk.services.s3.S3Client;

@RestController
@RequestMapping("/s3")
public class S3Controller {

    @Autowired
    private S3Client s3Client;

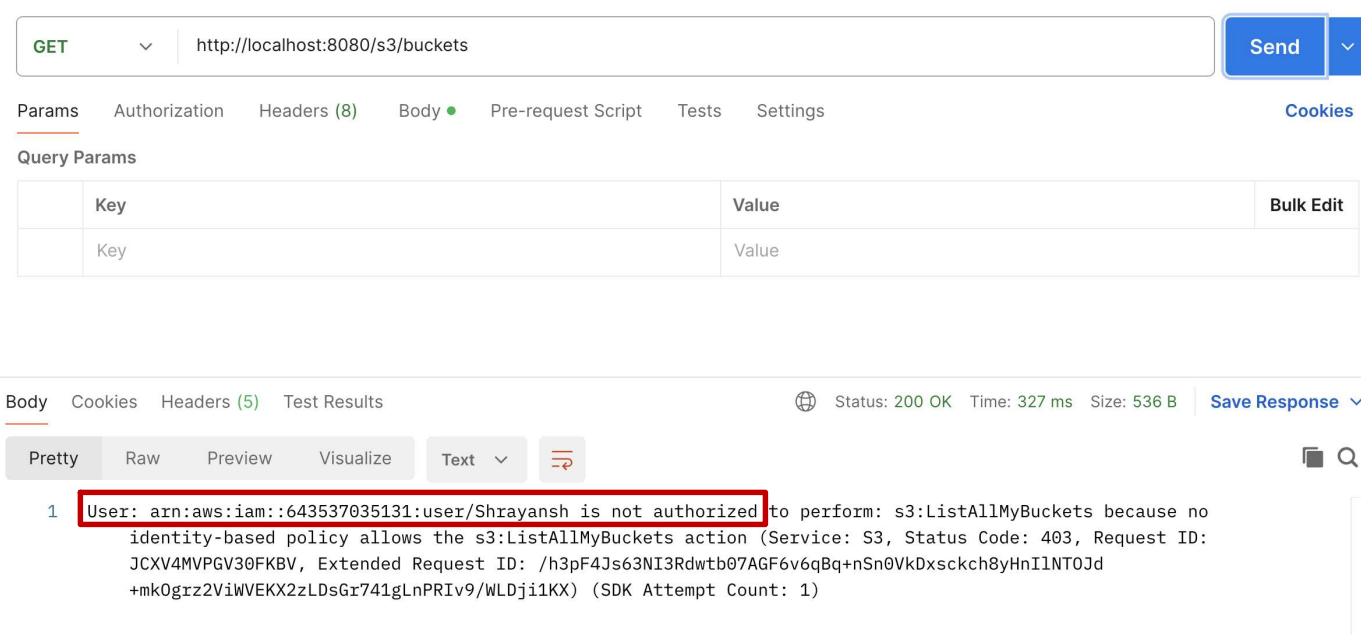
    @GetMapping("/buckets")
    public String listBuckets() {
        try {
            s3Client.listBuckets();
            return "SUCCESS";
        } catch (Exception e) {
            return e.getMessage();
        }
    }
}
```

Trying to access list of S3 buckets

application.properties

```
aws.access-key=ABCD...
aws.secret-key=MCTOz...
aws.region=ap-south-1
```

Postman call:



2. IAM Policy

Once AWS knows, WHO you are (Authentication via Username/Password or Access Key + Secret key)
Next it ask is "Are you allowed to do, what you are trying to do"?

Policy helps to answer this:

Its of 2 types:

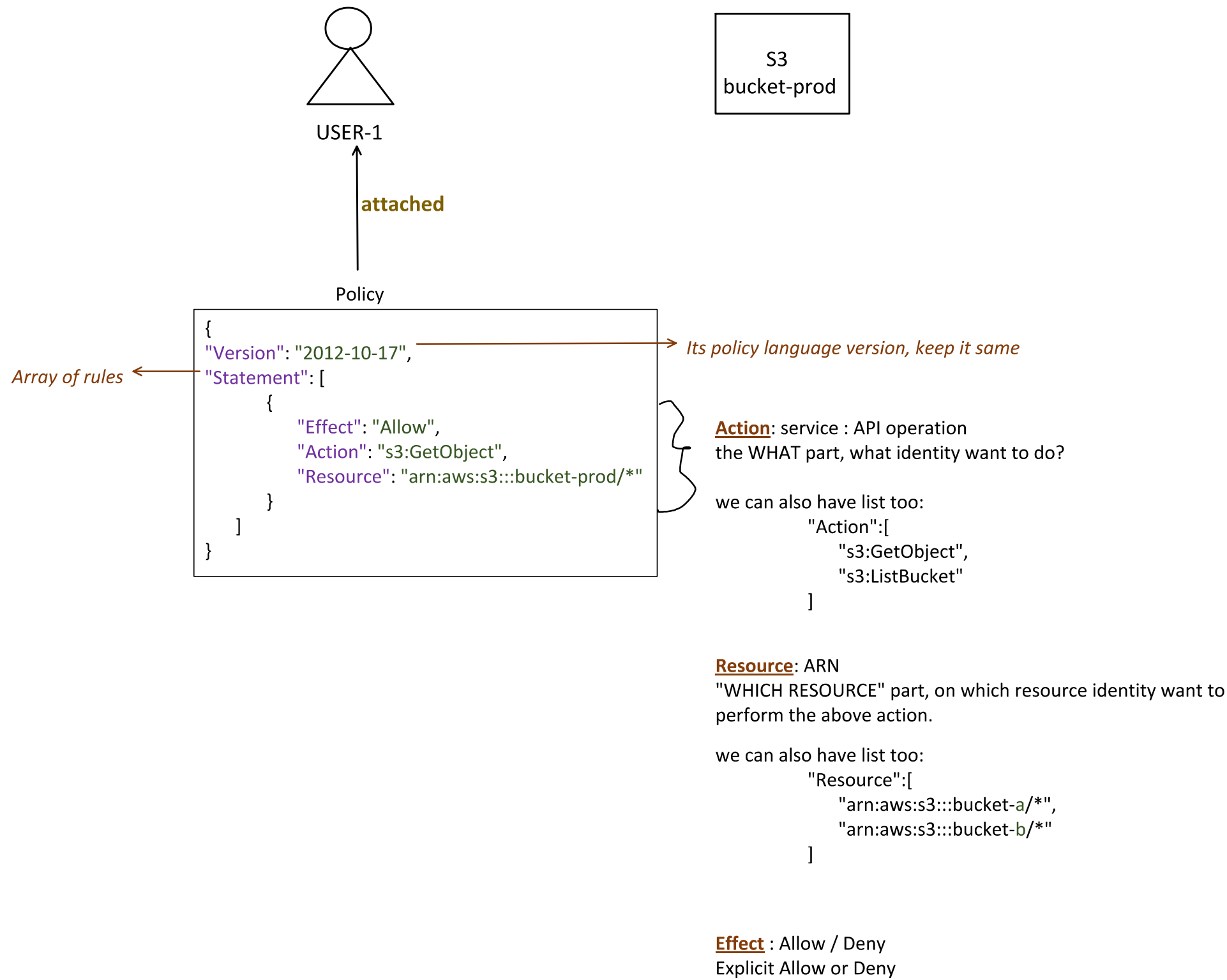
Identity Based Policies

Resource Based Policies

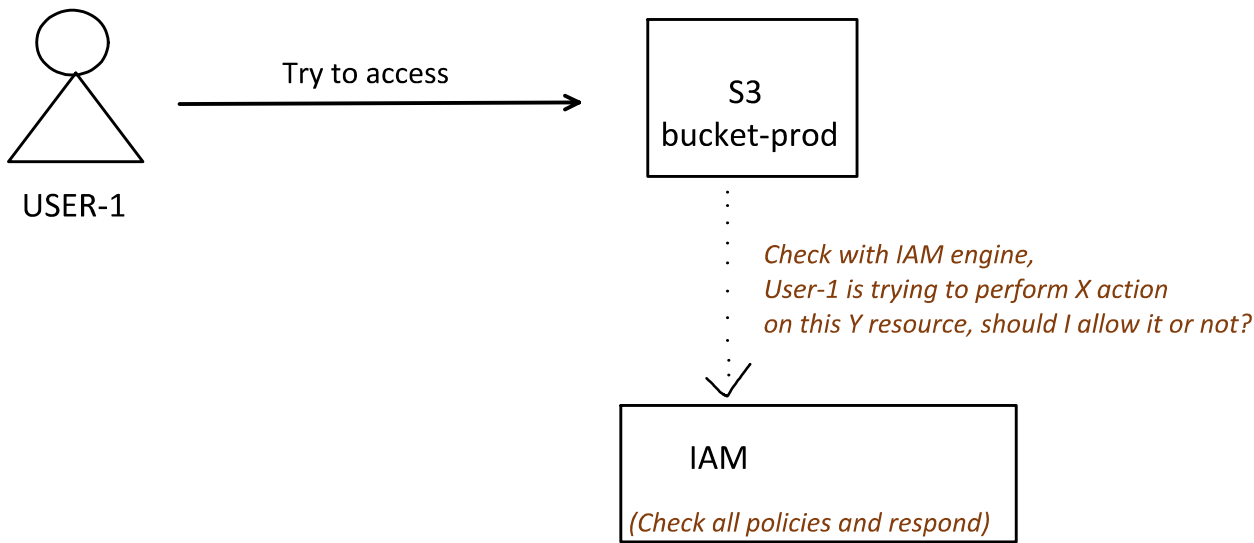
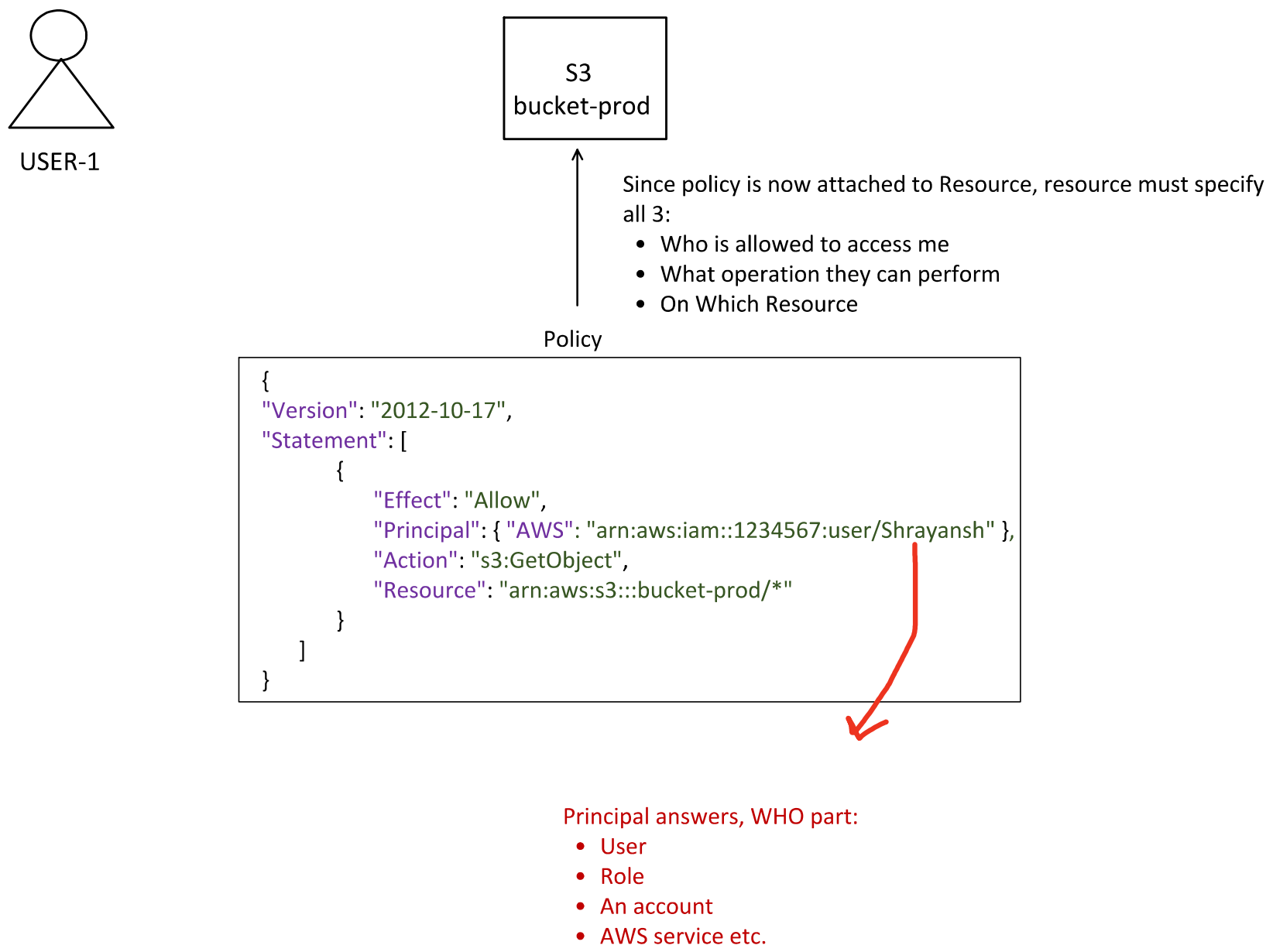
Both is same, but perceptive is different



Identity Based Policies : Policy is attached with IAM entity(User/Group/Role)

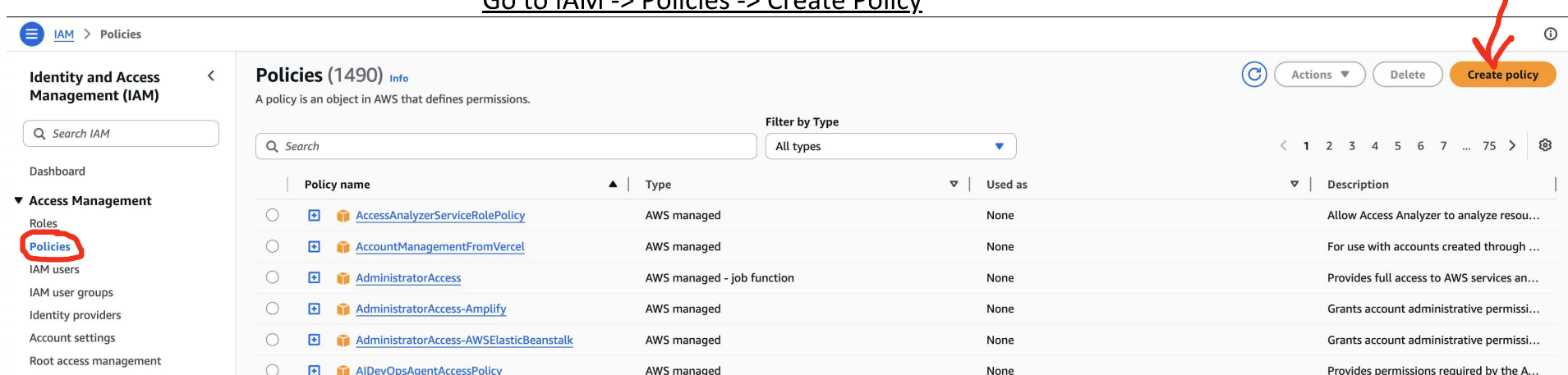


Resource Based Policies : Policy is attached with Resource

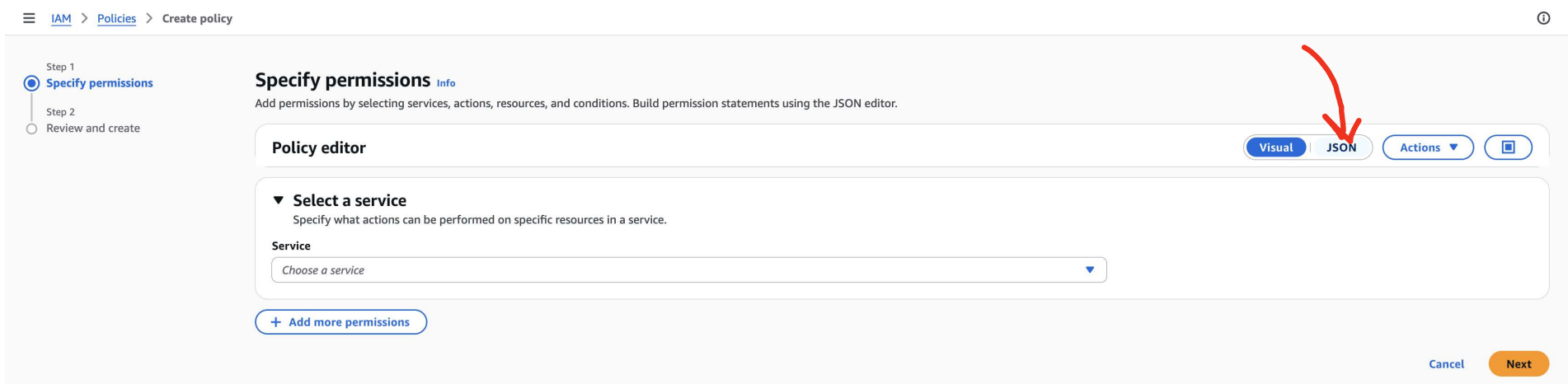


Lets Create Identity Based Policy:

Go to IAM -> Policies -> Create Policy



Click on JSON



Add Policy document and Click Next

Step 1
Specify permissions

Step 2
Review and create

Specify permissions

Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.

Visual JSON Actions

Policy editor

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": "s3:*",
7       "Resource": "*"
8     }
9   ]
10 }
```

Action -> S3: * means, S3 service and all its API (* wild card used)

Resource -> * means, all S3 resources (all buckets, all objects, all regions)

+ Add new statement

102 JSON

6016 of 6144 characters remaining

Cancel Next

Give proper policy Name -> Click Next

Step 1
Specify permissions

Step 2
Review and create

Review and create

Review the permissions, specify details, and tags.

Policy details

Policy name: AllowAllS3Actions

Description - optional

Permissions defined in this policy

Allow (1 of 469 services)

Service	Access level	Resource	Request condition
S3	Full access	All resources	None

Add tags - optional

Tags are key-value pairs that you can add to AWS resources to help identify, organize, or search for resources.

Cancel Previous Create policy

Now our policy is created.

Policy AllowAllS3Actions created.

Policies (1/491)

Filter by Type: All types

Policy name	Type	Used as
AllowAllS3Actions	Customer managed	None

Now lets attach these Policy to User

Go to IAM User -> Permissions tab

Identity and Access Management (IAM)

Shrayansh

Permissions

Permissions policies (0)

Generate policy based on CloudTrail events

Click Add permissions -> select Add Permissions

Shrayansh

Permissions

Add permissions

Permissions options

Attach policy directly

Permissions policies (1/1491)

AllowAll

AllowAllS3Actions

Next

Choose Attach policies directly -> search for our Custom policy -> select it

Step 1
Add permissions

Step 2
Review

Add permissions

Permissions options

Attach policy directly

Permissions policies (1/1491)

AllowAll

AllowAllS3Actions

Next

Click Add the Permissions

Step 1
Add permissions

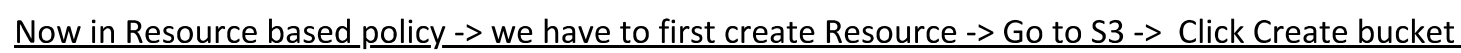
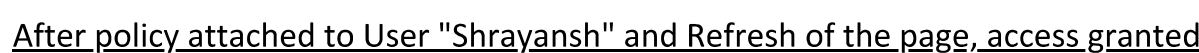
Step 2
Review

Review

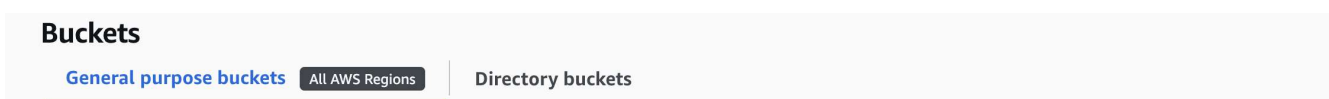
User details

User name: Shrayansh

Now, Our IAM User "Shrayansh" earlier getting Access Denied



Open the newly created Bucket (the resource)



General purpose buckets (1) info

Buckets are containers for data stored in S3.

Find buckets by name

< 1 >

Name	AWS Region	Creation date
sj-payment-prod	Asia Pacific (Mumbai) ap-south-1	May 28, 2026, 00:11:29 (UTC+05:30)

Click Permissions -> then Click Bucket Policy Edit

Amazon S3 > Buckets > sj-payment-prod

sj-payment-prod info

ObjectsMetadataPropertiesPermissionsMetricsManagementFile systems - newAccess Points

Permissions overview

Access finding

Block public access (bucket settings)

Block all public access

Individual Block Public Access settings for this bucket

Bucket policy

The bucket policy, written in JSON, provides access to the objects stored in the bucket. Bucket policies don't apply to objects owned by other accounts.

Public access is blocked because Block Public Access settings are turned on for this bucket

Add the policy.json document and Save

Bucket policy

The bucket policy, written in JSON, provides access to the objects stored in the bucket. Bucket policies i

Bucket ARN

arn:aws:s3:::sj-payment-prod

Policy

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Principal": {
7         "AWS": "arn:aws:iam::643537835131:user/Shravanish"
8       },
9       "Action": "s3:*",
10      "Resource": [
11        "arn:aws:s3:::sj-payment-prod",
12        "arn:aws:s3:::sj-payment-prod/*"
13      ]
14    }
15  ]
16 }
```

Permission for All S3 API operation

On bucket "sj-payment-prod" and On all objects within "sj-payment-prod"

Now lets test:

Controller class:

```
package com.concepts;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import software.amazon.awssdk.services.s3.S3Client;

@RestController
@RequestMapping("/s3")
public class S3Controller {
    @Autowired
    private S3Client s3Client;

    @GetMapping("/bucket-location")
    public String bucketLocation() {
        try {
            String bucketName = "sj-payment-prod";
            return s3Client.getBucketLocation(
                request -> request.bucket(bucketName)
            ).locationConstraintAsString();
        } catch (Exception e) {
            return e.getMessage();
        }
    }
}
```

Just trying to access the Region name of specific bucket (specific resource)

Postman:

GET http://localhost:8080/s3/bucket-location

ParamsAuthorizationHeaders (8)BodyPre-request ScriptTests

Query Params

BodyCookiesHeaders (5)Test Results

PrettyRawPreviewVisualizeText

1 ap-south-1

But Notice that in UI: we are seeing Access denied when we are clicking on S3 Bucket list

ap-south-1-console.aws.amazon.com/s3/buckets?region=ap-south-1

Amazon S3 > Buckets

Buckets

General purpose bucketsAll AWS RegionsDirectory buckets

General purpose buckets (0) info

Buckets are containers for data stored in S3.

Find buckets by name

NameAWS RegionCreation date

You don't have permissions to list buckets

Account snapshot info

View dashboard

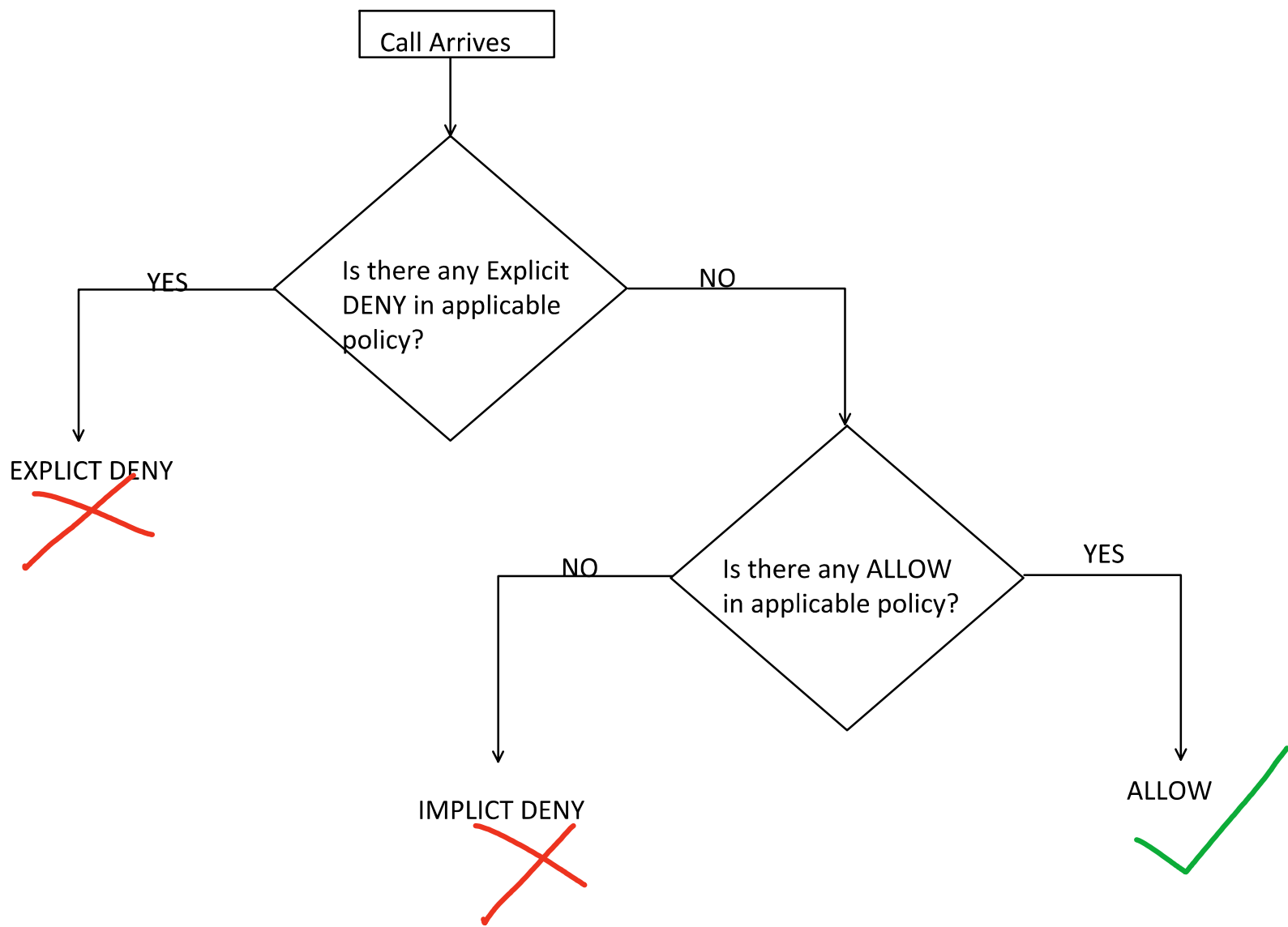
External access summary info

View dashboard

Its because, at this page (s3/buckets), AWS try to load all the buckets, not 1 specific resource (bucket) and our policy is attached with only 1 specific resource only.

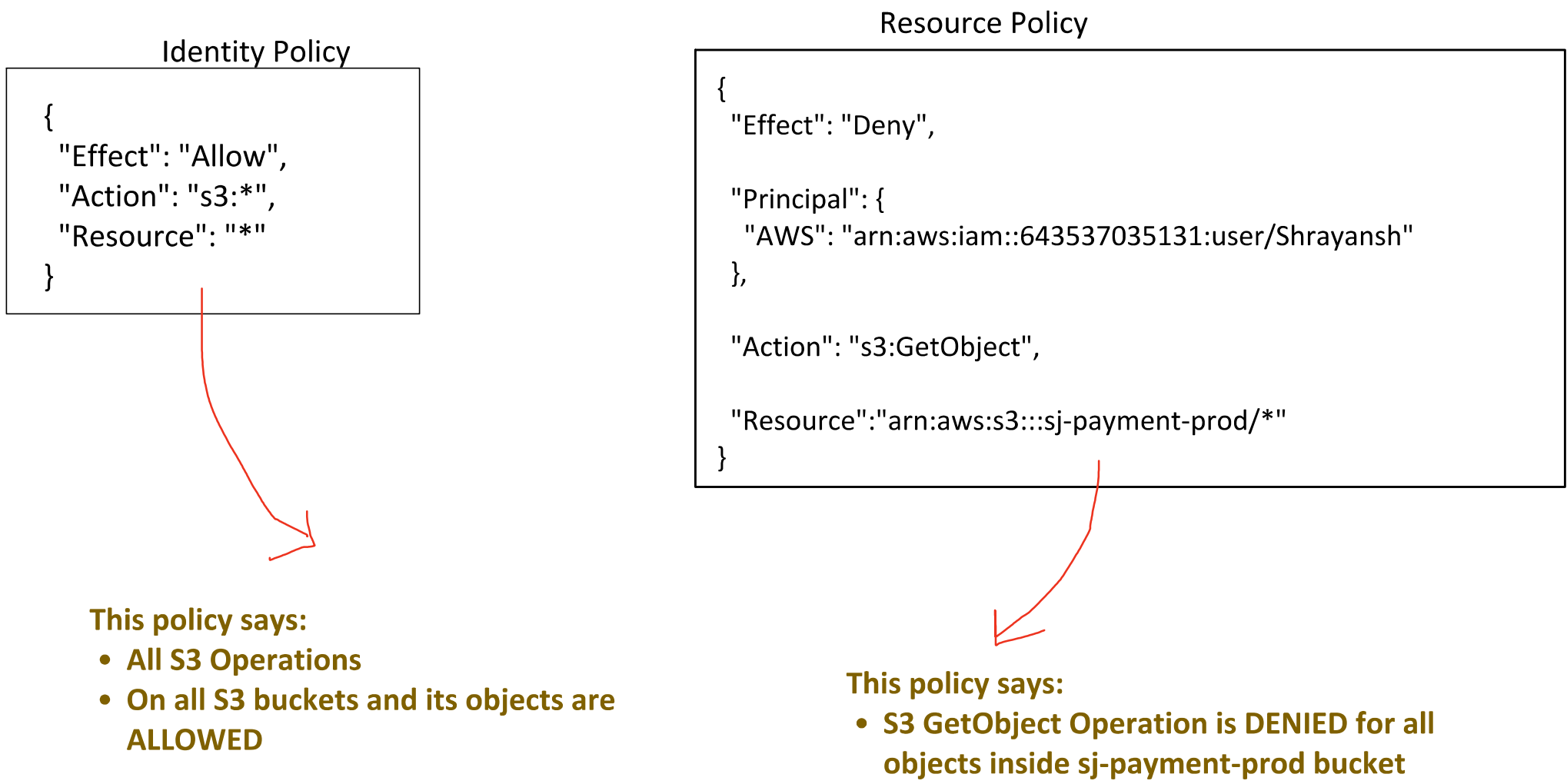
That's why with API, we can easily test by performing operation on specific resources, but through UI, we don't have much choice, so to test on UI, we might need to add Identity Based policy (for Action: S3:ListAllMyBuckets

The simplified flow:



Example:

Case 1 : Explicit Deny

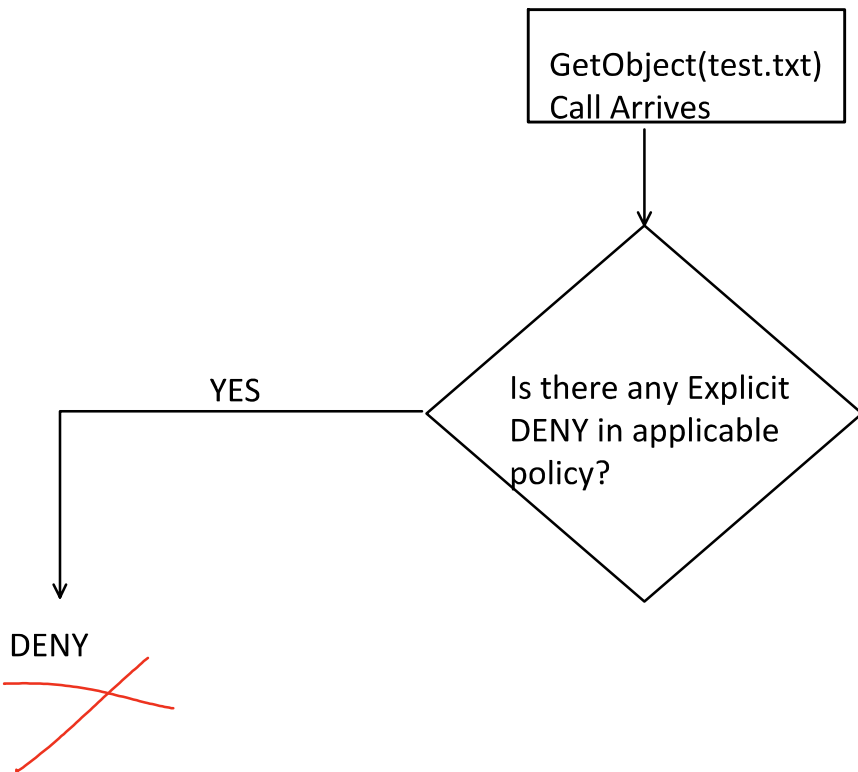


Now, lets say API call "GetObject(test.txt) on bucket sj-payment-prod comes.

- As per 1 policy:
- Identity policy : this operations is Allowed
- But as per 2nd policy:
- Resource policy: this operation is DENIED

What to do?

Check the flow:

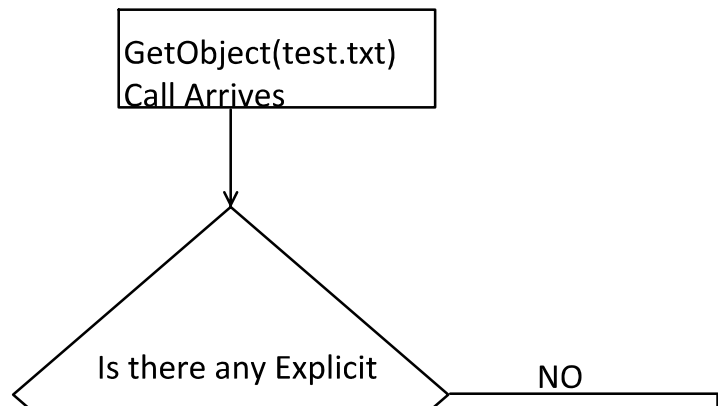


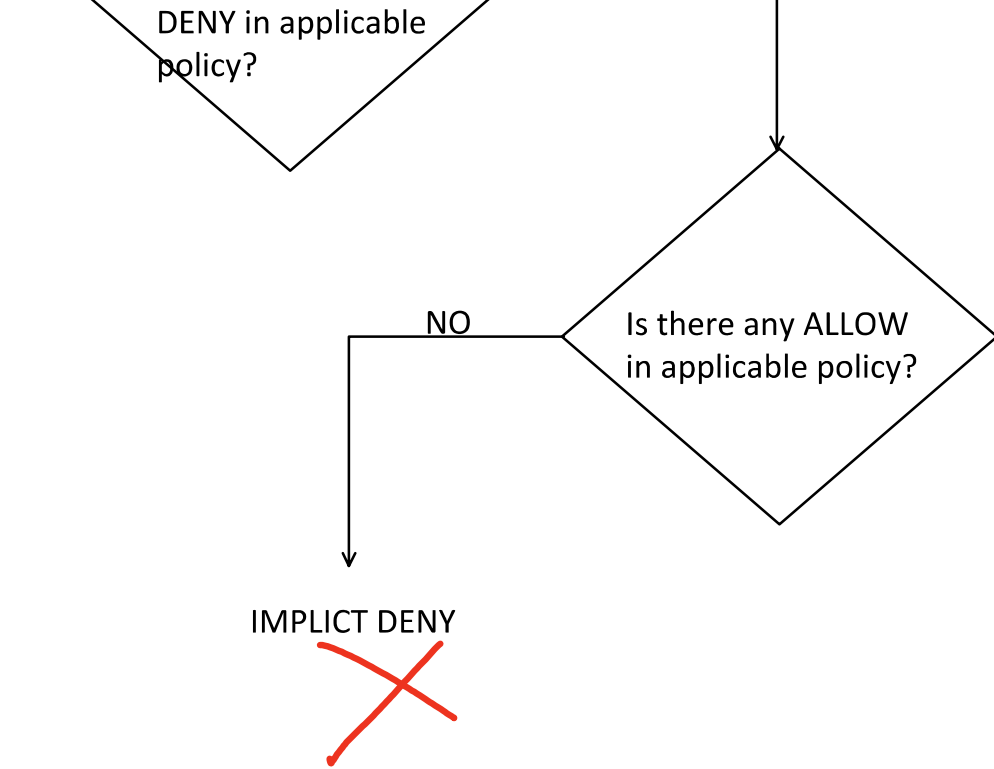
AWS club all the applicable polices for the flow:

Explicit DENY > Explicit Allow > Implicit Deny (default)

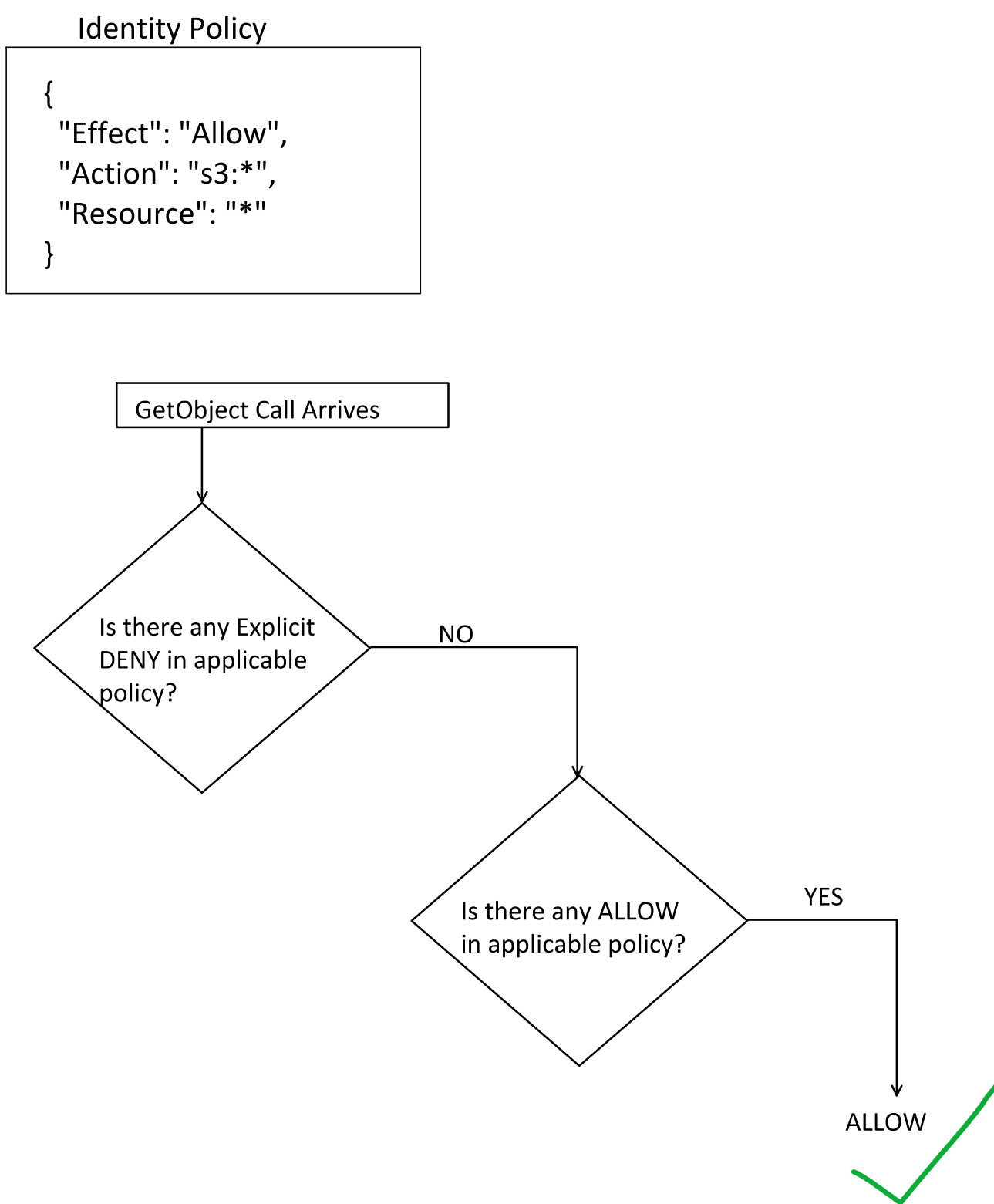
Case 2: Implicit Deny

There is no Policy exist itself





Case 3: Allow



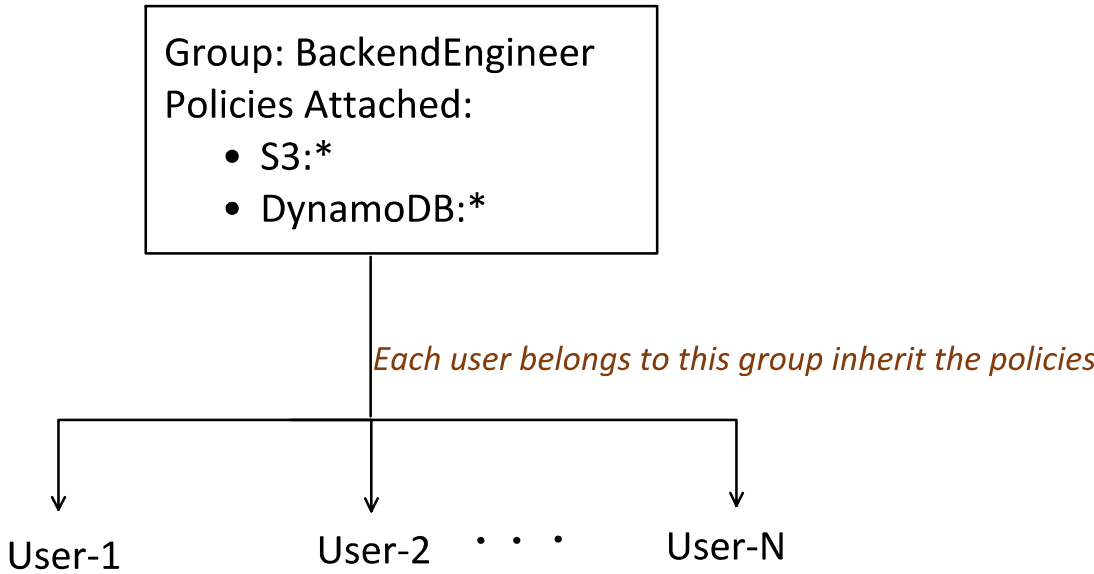
3. IAM Group

- Group is just a container for the User.
- Its not an Identity, means we can not sign in as Group or generate key for the group.

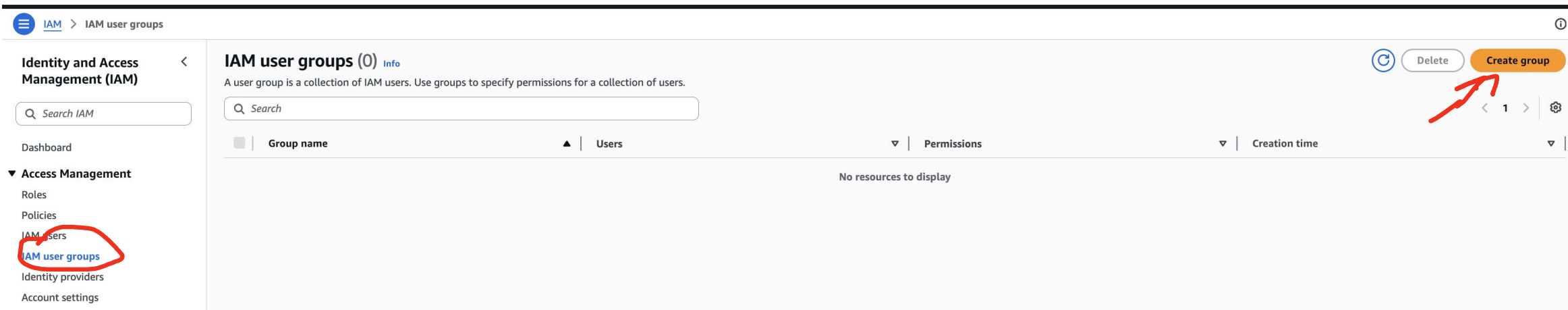
One main advantage of Group is:

- it can hold Policies that apply to every user in it.

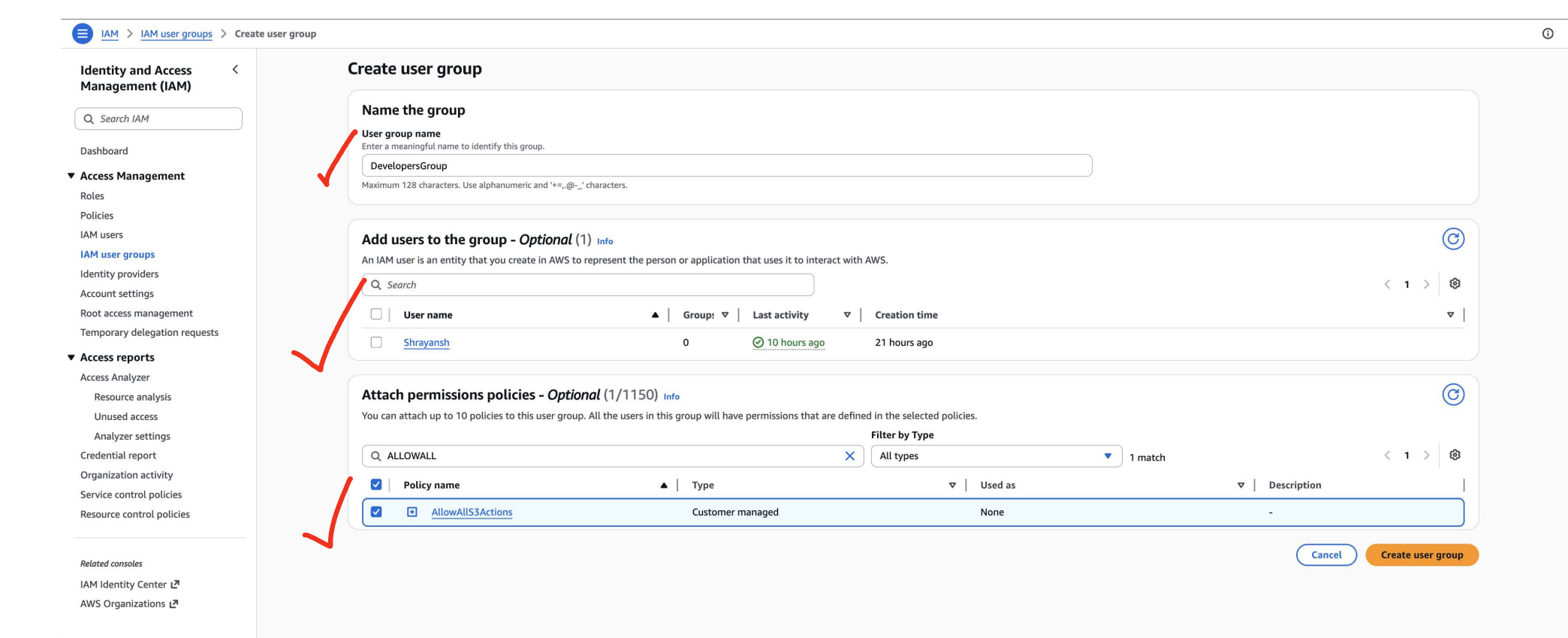
Ex: if we have 50 Users, we don't need to attach policy individually. We can create a Group and assign policy to Group (only once), and While creation of User, simply add User to specific Group.



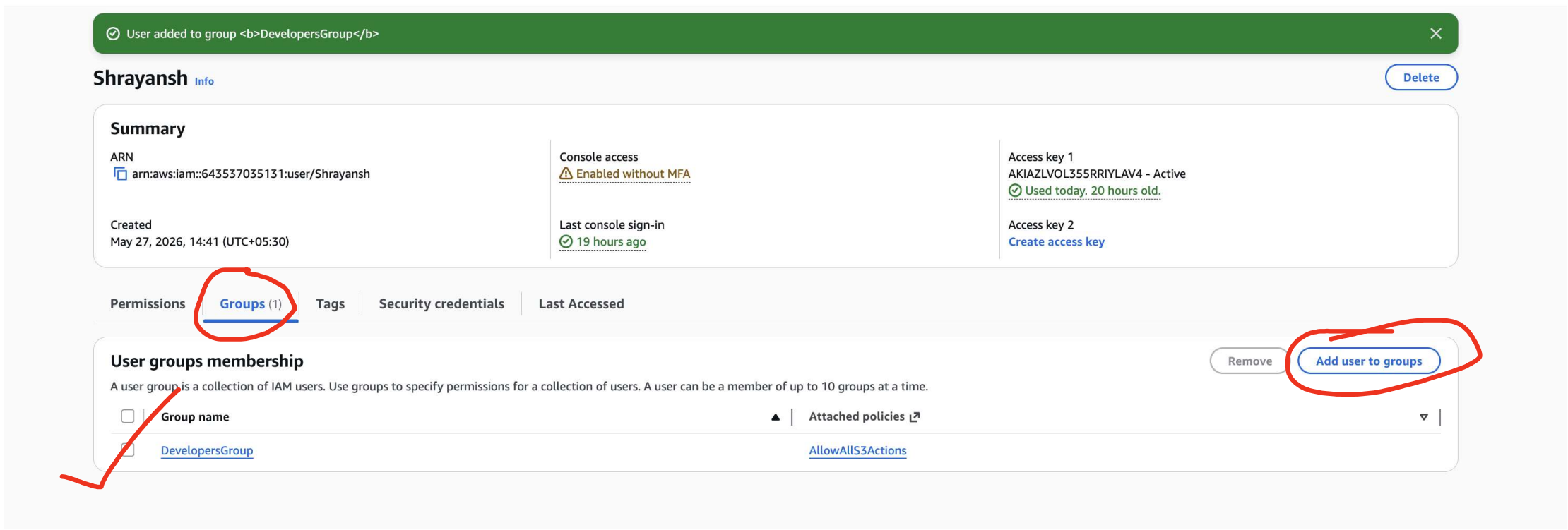
Go to IAM -> User Groups -> Create Group



Provide Group Name -> Option to Add Users to group -> Attach Policies to Group -> Click Create Group



If USER is not attached while Group Creation -> We can Go to IAM Users -> Select the USER -> Groups -> Add User to Group



4. IAM ROLE

IAM Role provides temporary identity.

Instead of storing long-lived AWS Access and Secret Keys (which can create security risks), Users/Services can ASSUME a Role and get temporary credentials.

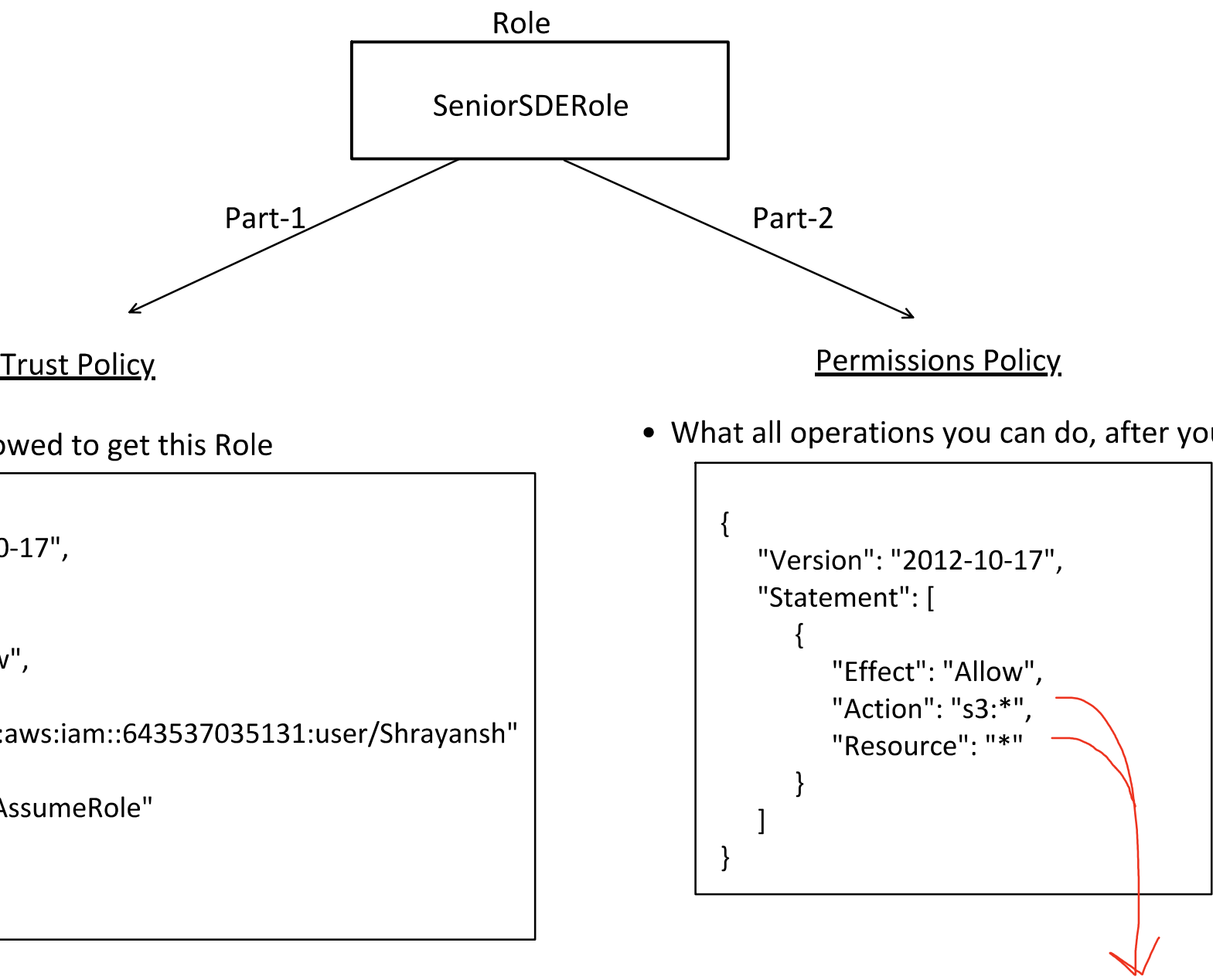
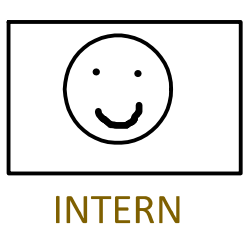
Example:

INTERN has READ only permission on S3.
SDE-3 has Read, Write permission on S3.
But
For 1 hour we need to provide INTERN all permissions with SDE-3 has

So how we can give?

Via ROLE.

- Create Role: SeniorSDERole
- Attach all the permission which SeniorSDE should have with this SeniorSDERole.
- Maintain the list of Identities which can Use this Role. Otherwise anyone can just use this Role, so we need some kind of whitelisting.



User Shrayansh belong to account 643537035131, is Allowed to invoked STS(Security Token Service) AssumeRole operation.

All S3 endpoints are allowed on All S3 resources (buckets, objects etc.)



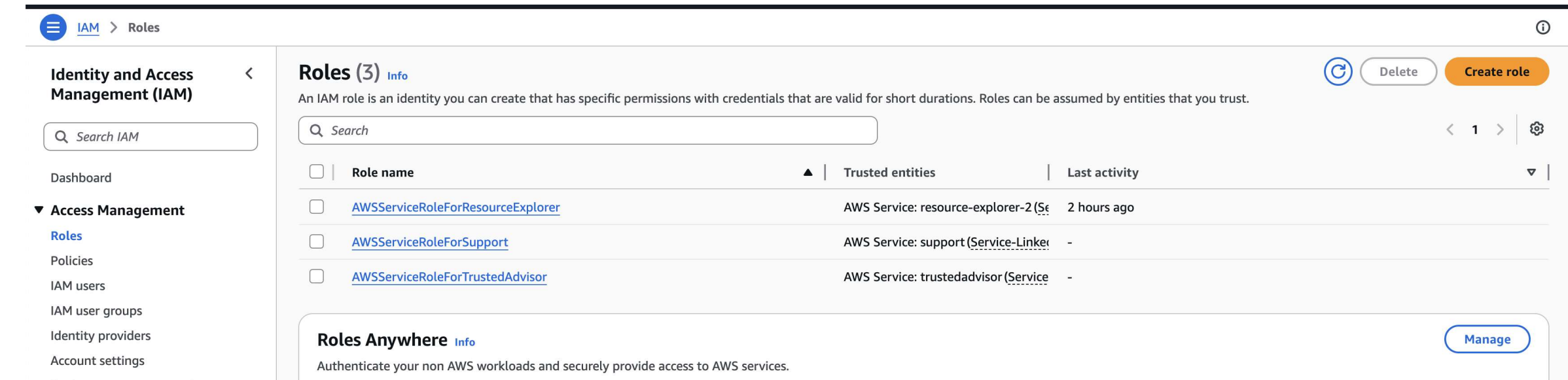
attached

Policy

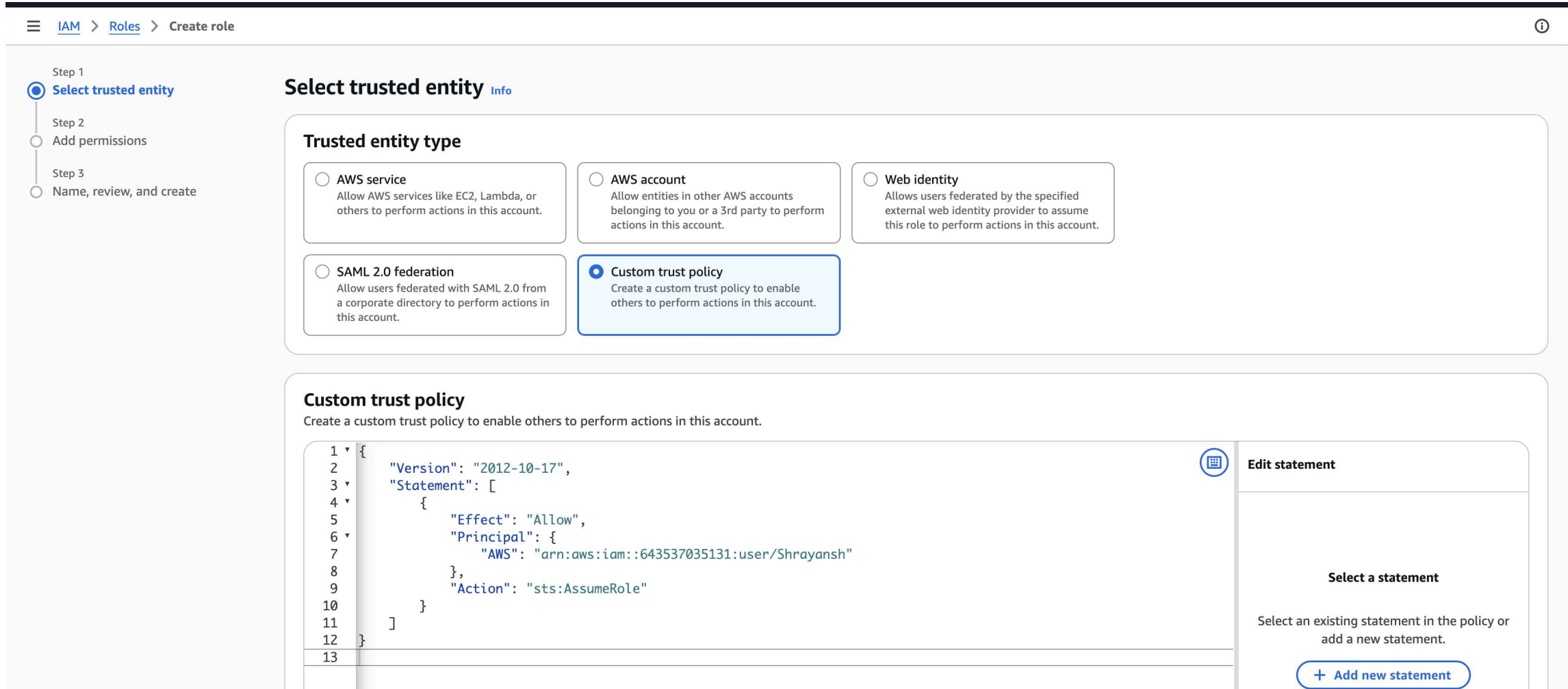
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "sts:AssumeRole",
      "Resource": "arn:aws:iam::643537035131:role/SeniorSDERole"
    }
  ]
}
```

Allow invocation of AssumeRole operation of STS service on given Role

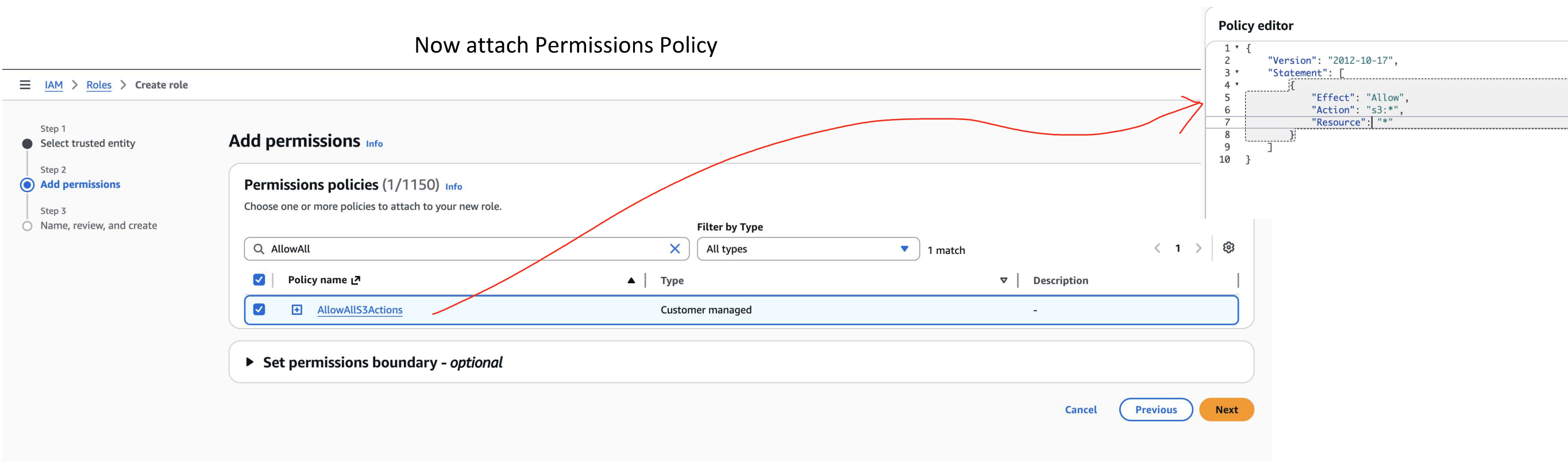
Got to IAM Roles -> Click Create Role



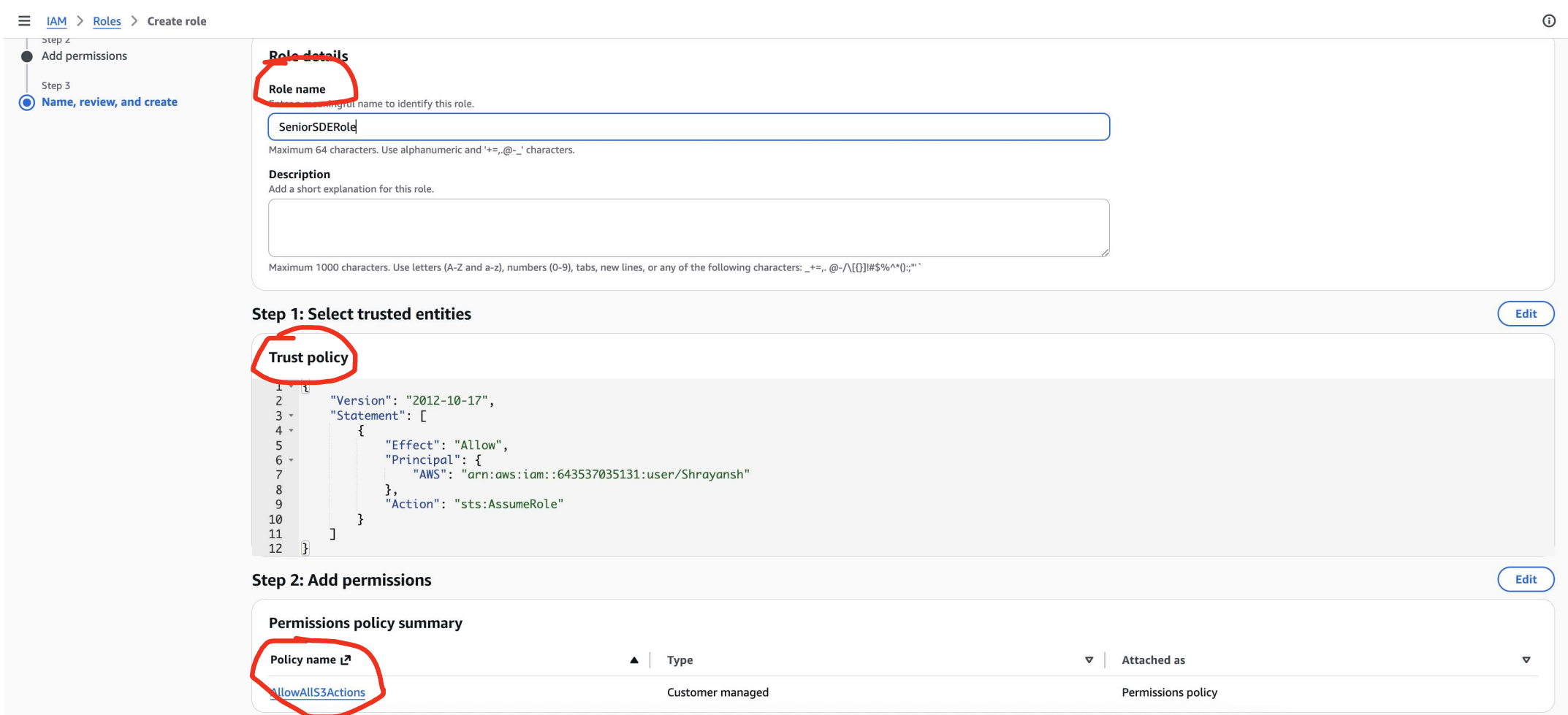
Click on Custom Trust Policy -> add the trust policy



Now attach Permissions Policy

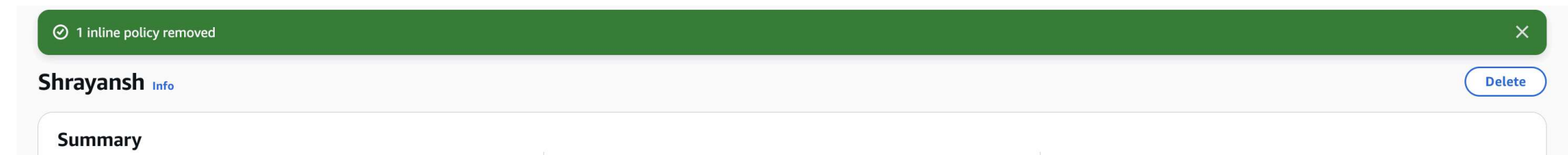


Provide Role Name and Review Trust Policy and Permissions attached



Now attach policy with USER

Go to IAM User -> Permissions -> Inline Policy



arn:aws:iam::643537035131:user/Shrayansh

Created
May 27, 2026, 14:41 (UTC+05:30)

Last console sign-in
Today

Access key 1
AKIAZLVOL355RRIRYLA4 - Active
Used today. Created yesterday.

Access key 2
Create access key

Permissions

Groups

Tags

Security credentials

Last Accessed

Permissions policies (0)

Permissions are defined by policies attached to the user directly or through groups.

Q Search

Filter by Type
All types

Policy name

Type

Attached via

No resources to display

Add Policy document

IAM > IAM users > Shrayansh > Create policy

Step 1
Specify permissions

Step 2
Review and create

Specify permissions

Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.

Policy editor

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": "sts:AssumeRole",
7       "Resource": "arn:aws:iam::643537035131:role/SeniorSDERole"
8     }
9   ]
10 }
```

Visual

JSON

Actions

Edit statement

Remove

Add actions

Choose a service

Q Search services

Included

STS

Give policy Name -> Create Policy

IAM > IAM users > Shrayansh > Create policy

Step 1
Specify permissions

Step 2
Review and create

Review and create

Review the permissions, specify details, and tags.

Policy details

Policy name
Enter a meaningful name to identify this policy.
AssumePolicyForSeniorSDERole
Maximum 128 characters. Use alphanumeric and "+", "@", "." characters.

Permissions defined in this policy

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it.

Q Search

Allow (1 of 469 services)

Service	Access level	Resource	Request condition
STS	Limited: Write	RoleName string like SeniorSDERole	None

Show remaining 468 services

Cancel

Previous

Create policy

Done

IAM > IAM users > Shrayansh

Identity and Access Management (IAM)

Dashboard

Access Management

Roles

Policies

IAM users

IAM user groups

Identity providers

Account settings

Root access management

Temporary delegation requests

Access reports

Access Analyzer

Resource analysis

Unused access

Analyzer settings

Credential report

Organization activity

Service control policies

Resource control policies

Policy AssumePolicyForSeniorSDERole created.

Shrayansh

Summary

ARN
arn:aws:iam::643537035131:user/Shrayansh

Console access
Enabled without MFA

Access key 1
AKIAZLVOL355RRIRYLA4 - Active
Used today. Created yesterday.

Created
May 27, 2026, 14:41 (UTC+05:30)

Last console sign-in
Today

Access key 2
Create access key

Permissions

Groups

Tags

Security credentials

Last Accessed

Permissions policies (1)

Permissions are defined by policies attached to the user directly or through groups.

Q Search

Filter by Type
All types

Policy name

Type

Attached via

AssumePolicyForSeniorSDERole

Customer inline

Inline

Permissions boundary (not set)

Now, Login as "Shrayansh" User which has this Role assigned and try to access S3

AWS

Search

[Option+S]

Account ID: 6435-3703-5131

Shrayansh

Amazon S3 > Buckets

Amazon S3

Buckets

General purpose buckets

Directory buckets

General purpose buckets (0)

Buckets are containers for data stored in S3.

Find buckets by name

Name	AWS Region	Creation date
------	------------	---------------

You don't have permissions to list buckets
After you or your AWS administrator has updated your permissions to allow the s3:ListAllMyBuckets action, refresh this page. Learn more about [identity and access management in Amazon S3](#)

Account snapshot
Updated daily
Storage Lens provides visibility into storage usage and activity trends.

External access summary
Updated daily
External access findings help you identify bucket permissions that allow public access or access from other AWS accounts.

Still Access Denied.

Its because, Shrayansh user has the ROLE, but it need to assume now (in other word, its need to invoke AssumeRole endpoint and then only it will get those permissions for limited period of time).

In UI how to invoke?

Go to your Role -> copy Link to switch roles in console

The screenshot shows the AWS IAM console for the 'SeniorSDERole'. The 'Summary' tab is active, displaying the role's ARN, creation date, and last activity. A red circle highlights the 'Link to switch roles in console' button, which provides a URL to switch to this role. Below the summary, the 'Permissions' tab shows the role has one policy attached: 'AllowAllS3Actions'.

Now, go to the browser where "Shrayansh" user is login and paste this link

Click on Switch Role

The 'Switch Role' dialog box is shown, allowing a user to switch to a different role. The 'IAM role name' field is filled with 'SeniorSDERole'. The 'Display name' and 'Display color' fields are optional and currently empty. The 'Switch Role' button is highlighted in orange.

And Now "Shrayansh" User has assumed SeniorSDERole for limited time

The screenshot shows the AWS S3 console with the 'Buckets' page. A red checkmark and the text 'S3 working fine' are visible. The 'Currently active as' dropdown menu is open, showing the user is signed in as 'Shrayansh' and has assumed the 'SeniorSDERole'.